Primer Proyecto de Inteligencia Artificial 2026

Detección de Ciberacoso en Redes Sociales

Opción B — Clasificación de Tweets

Ángel Esteban Esquit Hernández – 23221
Javier Eduardo España Pacheco – 23361
Roberto José Barreda Siekavizza – 23354

Abril 2026

Contents

1 Resumen Ejecutivo **3**

2 Presentación y Análisis de Datos **3**
2.1 Fuente y estructura del dataset . . . 3
2.2 Análisis exploratorio y distribución de clases . . . 3
2.3 Datos faltantes y atípicos . . . 4

3 Preprocesamiento de Texto (NLP Pipeline) **4**
3.1 Limpieza del texto . . . 4
3.2 Normalización . . . 5
3.3 Tokenización . . . 5
3.4 Eliminación de stopwords . . . 5
3.5 Lematización . . . 5
3.6 Ejemplo de transformación completa . . . 5

4 Vectorización: De Texto a Números **5**
4.1 Bag of Words (BoW) . . . 5
4.2 TF-IDF . . . 6
4.3 TF-IDF Híbrido (Palabras + Caracteres) . . . 6
4.4 TF-IDF Híbrido + Keywords Dirigidas . . . 6
4.5 Comparación de vectorizadores . . . 6

5 Partición de Datos y Metodología **7**
5.1 Partición train/test 7
5.2 Validación cruzada para tuning 7
5.3 Subconjunto estratificado para Random Forest 7

6 Modelos Probados y Configuración Técnica **7**
6.1 Naive Bayes Multinomial (Modelo Probabilístico) 7
6.2 Linear SVM — Support Vector Machine (Modelo Lineal) 7
6.3 Random Forest (Modelo de Ensamble) 8
6.4 Resumen de hiperparámetros 8

7 Resultados y Comparación de Modelos **8**
7.1 Métricas en conjunto de prueba 8
7.2 Métricas por clase del mejor modelo 9
7.3 Análisis train vs. test (overfitting) 9
7.4 Figuras de soporte 10

8 Matriz de Confusión y Análisis de Errores **11**
8.1 Confusiones más comunes 11
8.2 Ejemplos correctamente clasificados 12
8.3 Ejemplos incorrectamente clasificados 12

9 Justificación del Modelo Seleccionado **12**
9.1 Superioridad cuantitativa 12
9.2 Ventajas adicionales 12
9.3 Análisis de las clases difíciles 13

10 Pruebas de Usuario **13**

11 Conclusiones **14**

1 Resumen Ejecutivo

Este informe presenta el desarrollo de un sistema de clasificación supervisada para detectar ciberacoso en tweets. El problema se formula como clasificación multiclase sobre seis categorías: *age*, *ethnicity*, *gender*, *religion*, *other_cyberbullying* y *not_cyberbullying*. El flujo completo incluye preprocesamiento NLP, vectorización de texto (BoW, TF-IDF, TF-IDF híbrido y TF-IDF híbrido con keywords dirigidas), entrenamiento y comparación de tres modelos (Naive Bayes, Linear SVM y Random Forest), optimización de hiperparámetros mediante validación cruzada estratificada, análisis de overfitting y pruebas de usuario con una función interactiva de predicción.

El mejor modelo obtenido es **Linear SVM (tuned)** con un accuracy de 81.82% y un F1 ponderado de 0.8182 en el conjunto de prueba.

2 Presentación y Análisis de Datos

2.1 Fuente y estructura del dataset

Se utilizó el dataset *Cyberbullying Classification* disponible en Kaggle[1], almacenado localmente en `data/raw/cyberbullying_tweets.csv`. El archivo contiene 47,692 muestras con dos columnas: `tweet_text` (texto del tweet) y `cyberbullying_type` (etiqueta de clasificación).

2.2 Análisis exploratorio y distribución de clases

Se verificó la distribución de las seis clases, encontrando un balance prácticamente uniforme. La clase más numerosa es *religion* con 7,998 muestras y la menor es *other_cyberbullying* con 7,823; la diferencia es de apenas 175 muestras (0.37%), lo que permite enfocarse en el procesamiento del texto sin necesidad de técnicas de re-muestreo.

Table 1: Distribución de clases en el dataset (47,692 muestras totales)

Clase	Cantidad	Porcentaje
age	7,992	16.76%
ethnicity	7,961	16.69%
gender	7,973	16.72%
not_cyberbullying	7,945	16.66%
other_cyberbullying	7,823	16.40%
religion	7,998	16.77%

[1] https://www.kaggle.com/datasets/andrewmvd/cyberbullying-classification

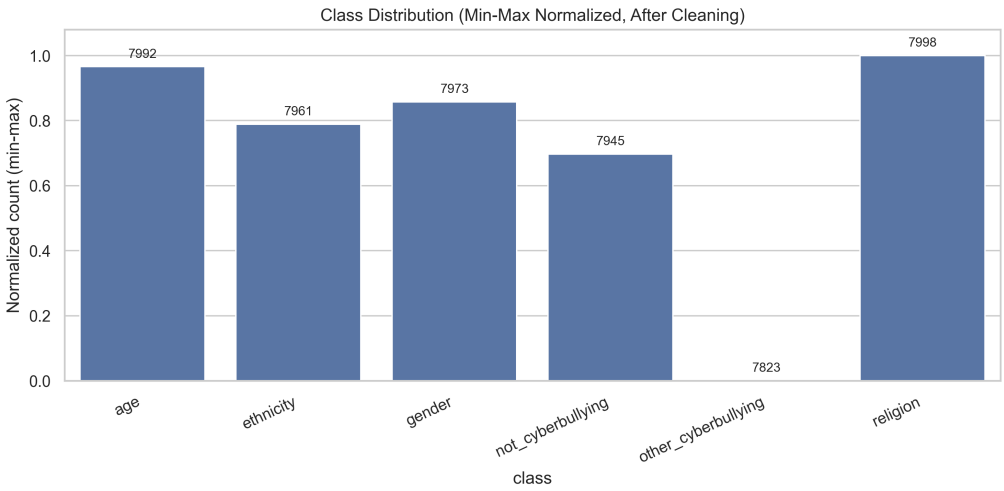

Figure 1: Distribución de clases normalizada (min-max) tras limpieza.

2.3 Datos faltantes y atípicos

Se realizó una inspección de calidad de datos. No se encontraron valores faltantes en ninguna de las dos columnas (`tweet_text` ni `cyberbullying_type`), por lo que las 47,692 filas se mantuvieron íntegras. El preprocesamiento de texto (Sección 3) se encarga de manejar datos atípicos a nivel textual, como tweets que contienen únicamente URLs o menciones y quedan vacíos tras la limpieza.

3 Preprocesamiento de Texto (NLP Pipeline)

El pipeline de preprocesamiento está implementado en `src/text_preprocessing.py` mediante la clase `TextPreprocessor`. Las etapas se aplican secuencialmente:

3.1 Limpieza del texto

- **Eliminación de URLs:** se remueven direcciones web (http, https, www) mediante expresiones regulares.
- **Eliminación de menciones** (`@usuario`): se eliminan por no correlacionarse con el tipo de ciberacoso.
- **Eliminación de hashtags** (`#`): se eliminan por completo; su formato concatenado dificulta la tokenización.
- **Eliminación de emojis y caracteres especiales:** se aplican patrones Unicode para emojis y luego se remueven todos los caracteres no alfanuméricos.

3.2 Normalización

Todo el texto se convierte a minúsculas y se colapsan espacios múltiples en un solo espacio.

3.3 Tokenización

Se utiliza `word_tokenize` de NLTK con la opción `preserve_line=True` para dividir cada tweet limpio en tokens.

3.4 Eliminación de stopwords

Se eliminan las stopwords del inglés (corpus de NLTK), con la excepción importante de conservar palabras de negación y pronombres personales: `no`, `nor`, `not`, `never`, `you`, `your`, `yours`, `yourself`, `yourselves`. Esta decisión es fundamental porque frases como "*no one wants you*" o "*you are not welcome*" perderían su significado de acoso si se eliminaran estas palabras.

3.5 Lematización

Se aplica el lematizador WordNet de NLTK para reducir cada token a su forma base (lema). Por ejemplo, "running" → "run" e "insulted" → "insult".

3.6 Ejemplo de transformación completa

> **Original:** *"Go back to your country, we do not want you here!!! @user123 #hate"*
> **Limpio:** *"go back your country not want you"*

4 Vectorización: De Texto a Números

Los modelos de aprendizaje automático requieren representaciones numéricas. Se implementaron cuatro esquemas de vectorización en `src/vectorization.py` y se compararon objetivamente con el mismo clasificador de referencia (LinearSVC, $C = 0.5$, `class_weight=balanced`).

4.1 Bag of Words (BoW)

Representa cada documento como un vector de conteos de frecuencia de unigramas y bigramas (`ngram_range=(1,2)`), limitado a 2,000 características (`max_features=2000`), con filtros `min_df=2` y `max_df=0.95`.

4.2 TF-IDF

Term Frequency – Inverse Document Frequency pondera cada término por su frecuencia local y su rareza global. Se emplea `sublinear_tf=True` para escala logarítmica ($1 + \log(\text{tf})$). Mismos parámetros de filtrado que BoW.

4.3 TF-IDF Híbrido (Palabras + Caracteres)

Combina dos vectorizadores TF-IDF concatenados horizontalmente (`hstack`): n-gramas de palabras (1,2) y n-gramas de caracteres a nivel de palabra (`char_wb`, rango 3–5, hasta 1,000 features). Los n-gramas de caracteres capturan patrones sub-léxicos como sufijos despectivos, variaciones ortográficas y jerga abreviada.

4.4 TF-IDF Híbrido + Keywords Dirigidas

Extiende el esquema híbrido con una tercera capa de características binarias basadas en un vocabulario fijo de 17 frases de rechazo y xenofobia (e.g. "go back your country", "not welcome here", "nobody want you"). Estas características se multiplican por un peso configurable (`keyword_feature_weight=3.0`), amplificando la señal de patrones indicadores de clases difíciles como *other_ cyberbullying*.

4.5 Comparación de vectorizadores

Table 2: Comparación de esquemas de vectorización (mismo clasificador LinearSVC)

Vectorizador	Accuracy	F1	Macro F1	Weak F1	Tiempo (s)
tfidf_hybrid	0.8184	0.8185	0.8175	0.5703	5.85
tfidf_hybrid_keywords	0.8182	0.8182	0.8173	0.5693	6.85
tfidf	0.8108	0.8114	0.8105	0.5630	1.44
bow	0.8069	0.8087	0.8077	0.5577	1.78

Los esquemas híbridos superan consistentemente a TF-IDF simple y BoW. Se seleccionó `tfidf_hybrid_keywords` como vectorizador de producción por su mayor robustez en escenarios de inferencia de usuario, donde las keywords dirigidas mejoran la clasificación de tweets con patrones explícitos de rechazo.

5 Partición de Datos y Metodología

5.1 Partición train/test

Se utilizó `train_test_split` de scikit-learn con partición 80/20 estratificada (`stratify=y`, `random_state=42`):

- **Entrenamiento:** 38,153 muestras (80%).
- **Prueba:** 9,539 muestras (20%).

5.2 Validación cruzada para tuning

Durante la búsqueda de hiperparámetros de Naive Bayes y Linear SVM se empleó `StratifiedKFold` con $k = 2$ folds sobre el conjunto de entrenamiento. Se optó por 2 folds para reducir el costo computacional, dado que cada fold contiene aproximadamente 19,000 muestras.

5.3 Subconjunto estratificado para Random Forest

Para controlar el costo computacional del modelo de ensamble, Random Forest se entrenó sobre un subconjunto estratificado de 12,000 muestras del conjunto de entrenamiento, reduciendo el tiempo de entrenamiento en aproximadamente un 68%.

6 Modelos Probados y Configuración Técnica

Se evaluaron tres arquitecturas, cada una representando una familia distinta de algoritmos.

6.1 Naive Bayes Multinomial (Modelo Probabilístico)

Clasificador probabilístico basado en el Teorema de Bayes con suposición de independencia condicional entre características. Eficiente para clasificación de texto al operar directamente sobre pesos TF-IDF.

Hiperparámetros ajustados:

- `alpha` (suavizado de Laplace): se probaron $\{0.005, 0.01, 0.05\}$. Mejor: `alpha=0.005`.
- Mejor F1 en validación cruzada: 0.7543.

6.2 Linear SVM — Support Vector Machine (Modelo Lineal)

`LinearSVC` implementa una SVM lineal optimizada para alta dimensionalidad. Busca el hiperplano que maximiza el margen de separación entre clases.

Hiperparámetros ajustados:

- `C` (regularización): se probaron $\{0.5, 1.0\}$. Mejor: $C = 0.5$.
- `class_weight`: se comparó `None` vs. `balanced`. Mejor: `balanced`.
- `max_iter`: 5,000 (garantizar convergencia).
- Mejor F1 en validación cruzada: 0.8109.

6.3 Random Forest (Modelo de Ensamble)

Construye 120 árboles de decisión sobre submuestras aleatorias (bagging). Cada árbol se entrena con un subconjunto aleatorio de características por nodo.

Hiperparámetros fijos (sin tuning por costo computacional):

- `n_estimators`: 120.
- `max_depth`: 20.
- `min_samples_split`: 2.
- Subconjunto de entrenamiento: 12,000 muestras estratificadas.

6.4 Resumen de hiperparámetros

Table 3: Hiperparámetros seleccionados y método de tuning

Modelo	Hiperparámetros	F1-CV	Método
Naive Bayes	`alpha = 0.005`	0.7543	Grid + CV-2
Linear SVM	`C=0.5, class_weight=balanced`	0.8109	Grid + CV-2
Random Forest	`n_est=120, depth=20, split=2`	N/A	Fijo

7 Resultados y Comparación de Modelos

7.1 Métricas en conjunto de prueba

Table 4: Comparación de modelos en test

Modelo	Accuracy	Precision	Recall	F1	Weak F1	Balance
Linear SVM (tuned)	0.8182	0.8187	0.8182	0.8182	0.5693	0.6938
Random Forest	0.7981	0.8189	0.7981	0.8024	0.5379	0.6702
Naive Bayes (tuned)	0.7606	0.7563	0.7606	0.7557	0.5083	0.6320

La métrica *balance_score* combina el rendimiento global con el de las clases más difíciles:

$$\text{balance_score} = 0.5 \times F1_{\text{ponderado}} + 0.5 \times \overline{F1}(\text{not_cb},\ \text{other_cb})$$

7.2 Métricas por clase del mejor modelo

Table 5: Reporte por clase: Linear SVM (tuned)

Clase	Precision	Recall	F1-Score	Support
age	0.9645	0.9850	0.9746	1,598
ethnicity	0.9731	0.9786	0.9759	1,592
gender	0.8775	0.8395	0.8581	1,595
not_cyberbullying	0.5598	0.5362	0.5477	1,589
other_cyberbullying	0.5738	0.6089	0.5908	1,565
religion	0.9574	0.9556	0.9565	1,600
Accuracy			**0.8182**	9,539
Macro avg	0.8177	0.8173	0.8173	9,539
Weighted avg	0.8187	0.8182	0.8182	9,539

7.3 Análisis train vs. test (overfitting)

Table 6: Comparación Train vs. Test por modelo (métricas principales)

Modelo	Métrica	Train	Test	Gap
Linear SVM (tuned)	Accuracy	0.8729	0.8182	0.0547
Linear SVM (tuned)	F1	0.8731	0.8182	0.0549
Linear SVM (tuned)	Balance score	0.7838	0.6938	0.0901
Naive Bayes (tuned)	Accuracy	0.7824	0.7606	0.0219
Naive Bayes (tuned)	F1	0.7785	0.7557	0.0228
Naive Bayes (tuned)	Balance score	0.6730	0.6320	0.0409
Random Forest	Accuracy	0.8275	0.7981	0.0294
Random Forest	F1	0.8321	0.8024	0.0297
Random Forest	Balance score	0.7227	0.6702	0.0525

Naive Bayes presenta el menor gap (F1 gap = 0.023), indicando que su simplicidad previene el sobreajuste pero también limita su rendimiento absoluto (*underfitting*). **Random Forest** muestra un gap intermedio (F1 gap = 0.030), y podría beneficiarse de más

datos de entrenamiento. **Linear SVM** tiene el mayor gap (F1 gap = 0.055), pero sigue siendo aceptable ya que se encuentra por debajo del umbral típico de preocupación (0.10), y sus métricas absolutas en test son las más altas.

7.4 Figuras de soporte

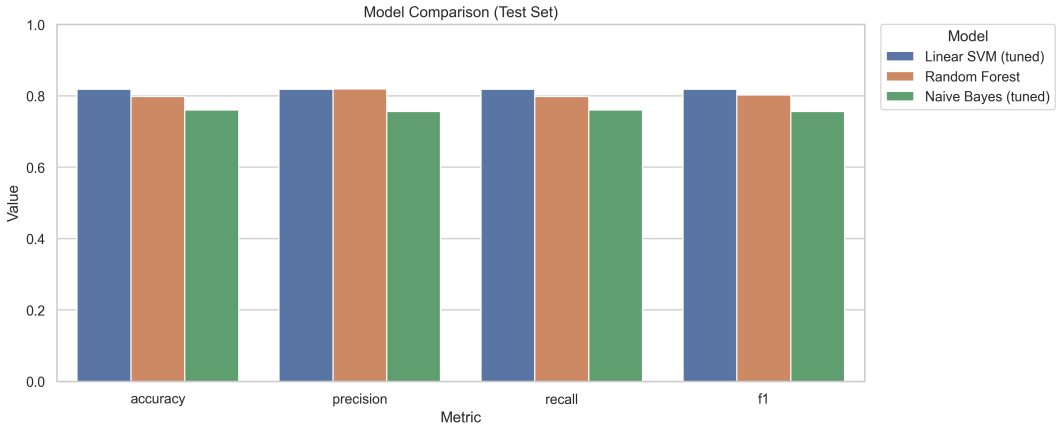

Figure 2: Comparación de métricas por modelo en el conjunto de prueba.

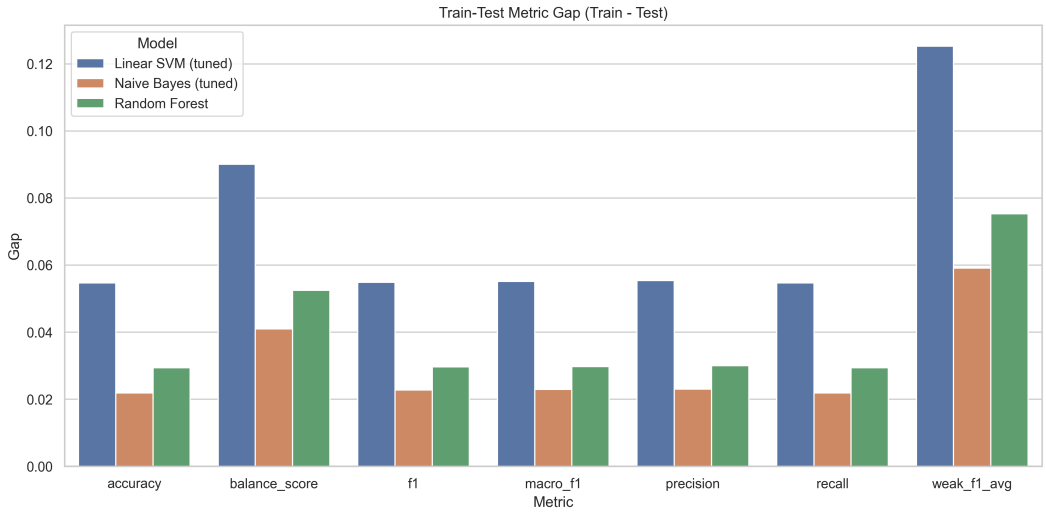

Figure 3: Brecha train-test por métrica y modelo.

8 Matriz de Confusión y Análisis de Errores

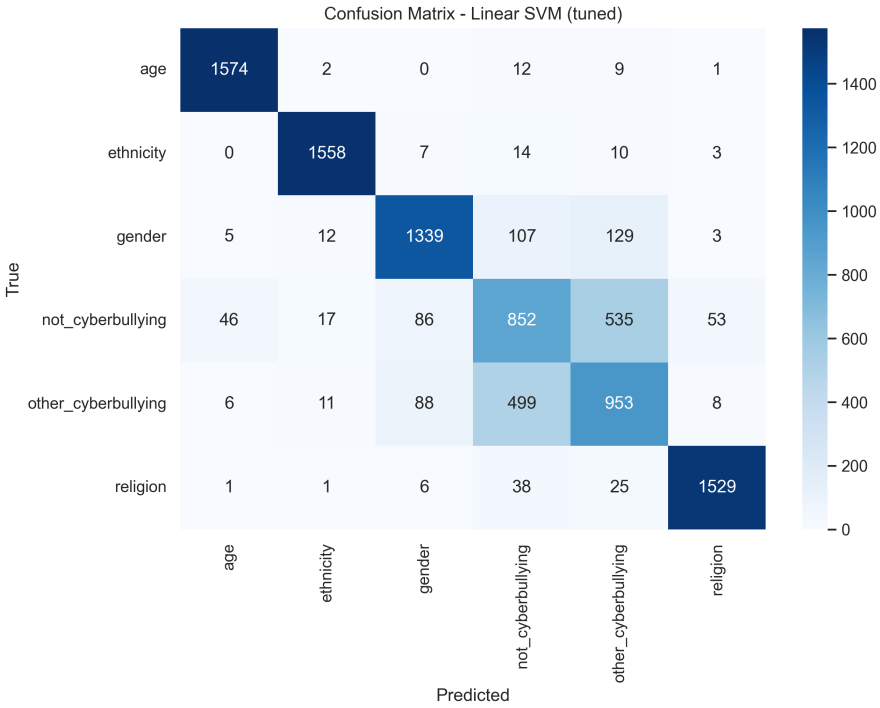

Figure 4: Matriz de confusión del Linear SVM (tuned) en el conjunto de prueba.

Las cuatro categorías de ciberacoso específico (*age, ethnicity, gender, religion*) se clasifican con alta precisión (recall $> 83\%$), mientras que *not_cyberbullying* y *other_cyberbullying* concentran la mayoría de los errores.

8.1 Confusiones más comunes

Table 7: Top 5 confusiones más frecuentes del mejor modelo

Clase real	Predicha como	Casos	Tasa de error
not_cyberbullying	other_cyberbullying	535	33.67%
other_cyberbullying	not_cyberbullying	499	31.88%
gender	other_cyberbullying	129	8.09%
gender	not_cyberbullying	107	6.71%
other_cyberbullying	gender	88	5.62%

La confusión bidireccional entre *not_cyberbullying* y *other_cyberbullying* ($535 + 499 =$ 1,034 errores) es el desafío principal, causada por la ambigüedad semántica inherente: el acoso genérico suele ser implícito, sarcástico o sutil y comparte características léxicas con tweets inofensivos.

8.2 Ejemplos correctamente clasificados

- *"you idiot death no time anyone you die tomorrow you idiot hatred muslim northerner..."* → **religion** (odio explícito hacia musulmanes).
- *"judge turn heat competition you think store remaining team"* → **not_cyberbullying** (lenguaje neutral).

8.3 Ejemplos incorrectamente clasificados

- *"vomit"* — Real: not_cyberbullying. Predicho: other_cyberbullying.
  Análisis: tweet de una sola palabra con connotación negativa; el modelo interpreta el término visceral como hostilidad.
- *"fact paid 7 fruit cup coffee school insane"* — Real: not_cyberbullying. Predicho: age.
  Análisis: la mención de "school" activa patrones de acoso por edad; el modelo no distingue una queja cotidiana.
- *"saying youre going put old yeller style triggering old yeller feeling"* — Real: gender. Predicho: other_cyberbullying.
  Análisis: contenido de género implícito; el modelo no capta la referencia cultural.

9 Justificación del Modelo Seleccionado

Se selecciona **Linear SVM (tuned)** como modelo final, basado en el criterio compuesto *balance_score* que combina el rendimiento global con el rendimiento en las clases más difíciles.

9.1 Superioridad cuantitativa

Linear SVM supera a los demás en todas las métricas principales (Tabla 4): accuracy 81.82% (> RF 79.81% > NB 76.06%), F1 ponderado 0.8182 (> 0.8024 > 0.7557) y balance_score 0.6938 (> 0.6702 > 0.6320).

9.2 Ventajas adicionales

- **Eficiencia computacional:** LinearSVC es extremadamente rápido. El pipeline completo se ejecutó en 40.15 segundos (semilla 42, 1 núcleo).
- **Escalabilidad:** opera eficientemente en el espacio disperso de alta dimensionalidad generado por TF-IDF, sin requerir reducción de dimensionalidad.

- **Regularización:** $C = 0.5$ con `class_weight=balanced` proporciona un buen equilibrio entre sesgo y varianza.
- **Interpretabilidad:** modelo lineal que facilita la explicabilidad, relevante en aplicaciones de moderación de contenido.

9.3 Análisis de las clases difíciles

Las categorías *not_ cyberbullying* y *other_ cyberbullying* son las más difíciles para todos los modelos. La rama de keywords dirigidas en la vectorización y la capa de reglas post-modelo en la función de usuario abordan parcialmente este desafío para escenarios de inferencia.

10 Pruebas de Usuario

El sistema incluye una función `predict_tweet` en `run_project.py` que recibe un texto libre, aplica preprocesamiento, vectoriza y retorna la predicción. Adicionalmente, se implementa una capa transparente de reglas post-modelo para mejorar la clasificación en escenarios de inferencia directa, sin afectar las métricas del benchmark.

Table 8: Resultados de la función de prueba de usuario

Tweet de entrada	Texto limpio	Predicción	Regla
I hope you have a great day!	hope you great day	not_cyberbullying	No
You are stupid and nobody wants you	you stupid nobody want you	other_cyberbullying	Sí
Go back to your country, we do not want you here	go back your country not want you	other_cyberbullying	Sí
I disagree with your opinion but respect your right to express it	disagree your opinion respect your right express	not_cyberbullying	Sí

La capa de reglas opera bajo tres condiciones: (1) **patrones seguros** (e.g. "respect your right") fuerzan *not_ cyberbullying*; (2) **patrones de alerta** (e.g. "go back your country") activan *other_ cyberbullying*; (3) **rechazo directo + insulto** (e.g. "not want you" + "stupid") activan *other_ cyberbullying*. Cuando una regla anula la predicción del modelo, se indica explícitamente en la salida.

11 Conclusiones

Linear SVM (tuned) fue seleccionado como mejor modelo con 81.82% de accuracy y F1 de 0.8182, superando a Random Forest y Naive Bayes en métricas globales y en las clases ambiguas; el principal desafío es la confusión bidireccional entre *not_ cyberbullying* y *other_ cyberbullying*, que concentra la mayoría de errores.